

Computation Offloading in NTN-empowered MEC using Multi-Agent Distributed Deep Reinforcement Learning

Nida Fatima

*Department of Computer Science
and Information Systems
BITS Pilani*

Hyderabad, India

p20190504@hyderabad.bits-pilani.ac.in

Paresh Saxena

*Department of Computer Science
and Information Systems
BITS Pilani*

Hyderabad, India

psaxena@hyderabad.bits-pilani.ac.in

Giovanni Giambene

*Department of Information Engineering
and Mathematical Sciences
University of Siena*

Siena, Italy

giovanni.giambene@unisi.it

Abstract—In this paper, we investigate Non-Terrestrial Network (NTN)-empowered Multi-access Edge Computing (MEC) systems for emerging applications like telemedicine, industrial automation, and augmented reality, which demand significant computational resources from User Equipments (UEs). To address these issues, we consider that UEs can either process tasks locally or offload them to edge servers deployed on Unmanned Aerial Vehicles (UAVs) and Low Earth Orbit (LEO) satellites. The efficacy of such systems crucially depends on formulating an efficient computation offloading policy. We propose a multi-agent distributed deep reinforcement learning (MADDRL) approach for computation offloading in NTN-empowered MEC systems. Our approach utilizes two distributed DRL frameworks: Centralized Training and Decentralized Execution (CTDE), where training occurs on a central controller, and Independent Learners (IL), where training is performed locally on the UEs. In both frameworks, policies are executed by UEs in a decentralized manner. Simulation results show that the CTDE framework reduces the average cost, defined as the weighted sum of delay and energy consumption experienced by the UEs, by 11%, 42%, and 66% compared to IL, local, and random policies, respectively.

Index Terms—Multi-agent distributed deep reinforcement learning, multi-access edge computing, non-terrestrial networks.

I. INTRODUCTION

The evolution of Internet of Things (IoT) has resulted in a considerable increase in the demand for delay-sensitive and computation-intensive applications. However, constrained by limited computing power, User Equipment (UEs) face challenges in meeting these application requirements. Multi-access edge computing (MEC), which brings computational resources closer to the UEs, facilitates computations with minimal delay and energy consumption by offloading UE's task to the edge server for processing [1]. Terrestrial MEC can support UEs; however, they suffer from certain vulnerabilities and challenges, including malfunctioning of terrestrial MECs and coverage issues due to fixed deployment [2]. To overcome these issues, MEC resources deployed within Non-Terrestrial Networks (NTN), which encompasses UAVs

(Unmanned Aerial Vehicles) and LEO (Low Earth Orbit) satellites, represents a promising solution [3].

Furthermore, developing an optimal offloading strategy is crucial for achieving effective performance. Previous studies on offloading primarily focused on task characteristics and device locations [4], [5]. However, they often ignore the load level of the edge servers, which refers to the total number of tasks offloaded to a single server. This factor significantly impacts system performance, as an overload can occur when multiple UEs offload tasks to a single server, potentially leading to increased processing delays and task drops due to missed deadlines. Additionally, the objective of the optimization problem also plays a crucial role in developing an efficient solution for offloading. While algorithms aimed to minimize delay have been proposed [5], [6], other studies focus solely on minimizing energy consumption [7], [8]. However, a realistic model needs to consider both delay and energy consumption while designing efficient offloading strategies.

In this paper, we design a computation strategy for NTN-empowered MEC systems aimed at minimizing cost, defined as the weighted sum of energy consumption and delay. The offloading decisions are based on factors such as the locations of UEs and UAVs, task characteristics, and the load levels of edge servers. These factors create a high-dimensional state space that grows with the number of UEs. To address this complexity, we employ Deep Reinforcement Learning (DRL) to find an optimal solution and extend it to Multi-Agent Distributed DRL (MADDRL) that enables multiple UEs to make decentralized decisions simultaneously. Our approach considers both Centralized Training and Decentralized Execution (CTDE) and Independent Learners (IL) frameworks [9]. In the proposed CTDE-based framework, a central controller trains a dueling Deep Q-Network (DQN) model using local information from UEs. The model parameters are then distributed to UEs for decentralized decision-making during execution. Conversely, in IL-based framework, each UE trains its own dueling DQN model using its local information, enabling distributed decision-making without a central controller

in NTN empowered MEC systems. The main contributions of this paper can be summarized as follows:

- We propose a computation offloading strategy for NTN-empowered MEC systems, formulating an optimization problem to minimize the cost, defined as the weighted sum of energy consumption and delay.
- We transform the problem into a decentralized partially observable Markov decision process (Dec-POMDP) and propose two MADDRL approaches including CTDE-based framework and IL-based framework that allow UEs to make distributed decisions based on their local information.
- We validate our approach by comparing it against baseline methods including local and random policies. We demonstrate its benefits and scalability across various simulation settings including number of UEs, task generation probability, task size, task deadline, penalties and weighting factors.

II. SYSTEM MODEL AND PROBLEM FORMULATION

In this article, we consider an NTN-empowered MEC system, that comprises of M UEs, N UAVs, and a LEO satellite constellation. Both the UAVs and the LEO satellites are equipped with edge servers. UEs are denoted by $\mathcal{M} = \{1, 2, \dots, M\}$, while edge nodes are denoted by $\mathcal{N} = \{1, 2, \dots, N, N+1\}$, with indexes $1, 2, \dots, N$ denote UAVs and $N+1$ representing the current LEO satellite covering the envisaged area. A discrete time-slotted system of period T , consisting of time slots $\mathcal{I} = \{1, 2, \dots, I\}$, is considered, each with a duration of Δ seconds.

A. Mobility Model

UEs are distributed in a rectangular area of L length and W width. Their position during a single time slot i are represented as $\mathbf{p}_m(i) = (x_m(i), y_m(i))$, where $m \in \mathcal{M}$. At the beginning of every time slot, UE $m \in \mathcal{M}$ moves from position $\mathbf{p}_m(i)$ to the new position $\mathbf{p}_m(i+1) = [x_m(i) + v * t_{mov} * \cos \theta(i), y_m(i) + v * t_{mov} * \sin \theta(i)]$, where v , t_{mov} , and $\theta(i) \in [0, 2\pi]$ denote the speed of the UE, time for movement, and the angle for direction, respectively. The speed and the time for movement are kept constant. However, the angle is chosen arbitrarily. UAVs are hovering above the UEs at a fixed altitude H , and their locations are denoted as $\mathbf{q}_n = (X_n, Y_n, H)$, $n \in \mathcal{N}$. A LEO satellite, positioned far above the UEs, process the offloaded task before its visibility time with the UE ends. Subsequently, it is replaced by another LEO satellite. In the study, we analyze the performance during the coverage of a single LEO satellite at a time.

B. Task Model

We assume that at the beginning of time slot i , each UE has a new task generated with a certain task generation probability P . The generated task is modeled as $W_m(i) = (D_m(i), s_m, \tau_m)$ where $D_m(i)$ (in bits) denotes the size of the task, s_m (in CPU cycles per bit) denotes the CPU cycles required to process a unit bit, and τ_m (in time slots) denotes the deadline,

i.e., the maximum tolerable delay. However, if the task is not completely processed by the end of time slot $X(i)$, where $X(i) = i + \tau_m - 1$, then it will be dropped.

C. Task Offloading Decision

Task processing locations are defined as $\mathcal{C} = \{0, \dots, N+1\}$, with $c \in \mathcal{C}$ denoting the location index. If $c = 0$ or $c = N+1$, the task will be processed either locally or at the LEO satellite, respectively. If $c \in [1, N]$, the task are offloaded to the corresponding UAV. Let the binary variable $\alpha_{m,c}(i) \in \{0, 1\}$, indicates the offloading decision, where $\alpha_{m,c}(i) = 1$ indicates that at the time slot i , the task of m^{th} UE is processed at c^{th} location. Furthermore, as the task can only be processed at one location, we have, $\sum_{c \in \mathcal{C}} \alpha_{m,c}(i) = 1, \forall m \in \mathcal{M}, i \in \mathcal{I}$.

D. Communication Model

The transmission rate between UE m and UAV n ($r_{m,n}(i)$) and the transmission rate between UE m and satellite s ($r_{m,s}(i)$) are determined as follows:

$$r_{m,n}(i) = B_{m,n} * \log_2(1 + (P_{up,n} * h_{m,n}(i) / \sigma^2)); \quad (1)$$

$$r_{m,s}(i) = B_{m,s} * \log_2(1 + (P_{up,s} * |g|^2 / \sigma_s^2)) \quad (2)$$

where $B_{m,n}$, $B_{m,s}$ denote the communication bandwidth, $P_{up,n}$, $P_{up,s}$ denotes the uplink transmission power, σ^2 , σ_s^2 denotes the noise power, $h_{m,n}(i)$, g denotes the channel gain between UE m and UAV n , and between UE m and satellite s , respectively. $h_{m,n}(i)$ is calculated as $h_{m,n}(i) = \beta_0 * d_m^{-j}(i) = \beta_0 / \{\|\mathbf{q}_n - \mathbf{p}_m(i)\|^2 + H^2\}$. The value of $r_{m,s}$ (in Mbps) is assumed to be a constant, which is usually less than the data transfer rate of the UE-UAV link $r_{m,n}(i)$ [10].

E. Computation model

Each UE maintains two queues: computation and transmission. At the beginning of the time slot i , if the UE opts for local processing, the task enters the computation queue; otherwise, it enters the transmission queue for offloading and then it will be sent to the chosen edge node. Similarly, each edge node maintains M queues, each corresponding to a UE in $\mathcal{M}$. If UE m decides to offload its task, the offloaded task arrives at its respective queue at edge node n at the beginning of the time slot i^* where $i^* \in \{i+1, \dots, I\}$ and $i^* > i$.

For a task generated at UE m at the beginning of time slot i , if $\alpha_{m,c}(i) = 1$ where $c = 0$ then local processing is performed else if $\alpha_{m,c}(i) = 1$ where $c \in \{1, N+1\}$, then edge processing is performed. The local processing delay $t_{local,m}(i)$ (in time slots), the transmission delay $t_{tran,m}(i)$ (in time slots) and the edge processing delay $t_{m,n}^{edge}(i)$ (in time slot) are determined as follows:

$$t_{local,m}(i) = l_m^{comp}(i) - i + 1, \quad (3)$$

$$t_{tran,m}(i) = l_m^{tran}(i) - i + 1, \quad (4)$$

$$t_{edge,m}(i) = l_{m,n}^{edge}(i^*) - (i^*) + 1, \quad (5)$$

where $l_m^{comp}(i)$, $l_m^{tran}(i)$ and $l_{m,n}^{edge}(i^*)$ denote the time slot when the task is locally processed or dropped, transmitted or

dropped and processed at edge or dropped, respectively. They are calculated as:

$$l_m^{comp}(i) = \min\{i + \delta_m^{comp}(i) + \gamma_m^{comp}(i) - 1, X(i)\}, \quad (6)$$

$$l_m^{tran}(i) = \min\{i + \delta_m^{tran}(i) + \gamma_m^{tran}(i) - 1, X(i)\}, \quad (7)$$

$$l_{m,n}^{edge}(i^*) = \min\{i^* + \delta_{m,n}^{edge}(i^*) + \gamma_{m,n}^{edge}(i^*) - 1, X(i)\}, \quad (8)$$

where the first term in the min operator is the time slot when the processing or transmission is completed. The second term is the time slot when the task will be dropped. The waiting time before local processing, transmission and edge processing starts are denoted by $\delta_m^{comp}(i)$, $\delta_m^{tran}(i)$ and $\delta_{m,n}^{edge}(i^*)$, respectively. They are determined by:

$$\delta_m^{comp}(i) = \max_{i' \in \{0,1,\dots,i-1\}} l_m^{comp}(i') - i + 1, \quad (9)$$

$$\delta_m^{tran}(i) = \max_{i' \in \{0,1,\dots,i-1\}} l_m^{tran}(i') - i + 1, \quad (10)$$

$$\delta_{m,n}^{edge}(i^*) = \max_{i' \in \{0,1,\dots,i^*-1\}} l_{m,n}^{edge}(i') - i^* + 1, \quad (11)$$

where the first term on the right hand side of the equations determines the time slot when all the tasks placed in the computation queue for local processing, transmission queue for offloading and queue at the edge node n for edge processing has either been processed or dropped.

Finally, $\gamma_m^{comp}(i)$, $\gamma_m^{tran}(i)$, $\gamma_{m,n}^{edge}(i^*)$ denotes the local processing time, transmission time and edge processing time, respectively. For local processing time, $\gamma_m^{comp}(i) = [(D_m(i) * s_m) / (f_m^{device} * \Delta)]$, where f_m^{device} (in CPU cycles per second) denotes the processing capacity of the CPU at UE m , $D_m(i)$ and s_m represents the task size, and processing density, respectively and Δ is the duration of time slot in seconds.

For transmission time, $\gamma_m^{tran}(i) = [D_m(i) / (r_{m,n}(i) \Delta)]$ if the task is offloaded to UAV n or $\gamma_m^{tran}(i) = [(D_m(i) / (r_{m,s}(i) \Delta)) + T_p / \Delta]$ if the task is offloaded to satellite s , where $r_{m,n}(i)$, $r_{m,s}(i)$ denote the transmission rate between UE m and UAV n or LEO satellite s respectively. However, in the case of LEO satellite, the propagation time T_p is also considered.

For edge processing time, to evaluate the value of $\gamma_{m,n}^{edge}(i^*)$ it is crucial to find the processing capacity of the edge node which is equally shared among all the offloaded tasks. This division can be determined by knowing the number of active queues $B_n(i^*)$, where a queue at the edge node is considered active if it has a new task arrival or if it is non empty at the end of previous slot ($q_{m,n}^{edge}(i^* - 1) > 0$, where $q_{m,n}^{edge}(i^* - 1)$, $m \in \mathcal{M}$ denotes the queue length at the end of the time slot $i^* - 1$. Therefore, processing time at the edge node is given by $\gamma_{m,n}^{edge}(i^*) = [D_m(i) * s_m / (f_n^{edge} / B_n(i^*)) * \Delta]$ where f_n^{edge} (in CPU cycles per second) denotes the processing capacity of the CPU of the edge node $n \in \mathcal{N}$. The queue length is updated at the end of time slot i^* using $q_{m,n}^{edge}(i^*) = (q_{m,n}^{edge}(i^* - 1) + D_m(i^*) - ((f_n^{edge} \Delta / B_n(i^*)) / s_m) - e_{m,n}^{edge}(i^*))$, where $q_{m,n}^{edge}(i^* - 1)$ denotes the previous length, $D_m(i^*)$, $f_n^{edge} \Delta / B_n(i^*) / s_m$, $e_{m,n}^{edge}(i^*)$ denotes the number of bits arrived, processed and dropped, respectively.

The energy consumption of UE m for local computation $e_{local,m}(i)$ and for transmission $e_m^{tran}(i)$ are calculated as:

$$e_{local,m}(i) = \kappa * D_m(i) * s_m * (f_m^{device} * \Delta)^2, \quad (12)$$

$$e_m^{tran}(i) = (P_{up} * D_m(i)) / (r_m(i) \Delta), \quad (13)$$

where κ is the effective switched capacitance coefficient, P_{up} is $P_{up,n}$, $P_{up,s}$ and $r_m(i)$ is $r_{m,n}(i)$, $r_{m,s}(i)$ for UAV and LEO satellite, respectively.

F. Problem Formulation

We formulate the optimization problem to minimize the cost of UE m by optimizing its task offloading decision. The cost at the time slot i for the UE m is denoted by $\Psi_m(i)$. It is the weighted summation of delay $t_m(i)$ and energy consumption $e_m(i)$. It is defined as follows:

$$\Psi_m(i) = w_1 t_m(i) + w_2 e_m(i), \quad (14)$$

where $w_1, w_2 \in [0, 1]$ such that $w_1 + w_2 = 1$ denote the weighted factors to indicate the significance of delay and energy consumption, respectively. $t_m(i)$ denotes the delay experienced by the UE, where $t_m(i)$ is $t_{local,m}(i)$, $t_{tran,m}(i) + t_{edge,m}(i)$ and $e_m(i)$ is $e_{local,m}(i)$, $e_{tran,m}(i)$ for local computation and edge computation, respectively. The objective is to minimize the average cost of each UE m for all the tasks generated by it over I time slots. Thus, the computation offloading problem is formulated as follows:

$$\min_{\{\alpha_{m,c}(i)\}} \lim_{I \rightarrow \infty} \frac{1}{I} \sum_{i=1}^I \sum_{c=0}^{N+1} \alpha_{m,c}(i) (\Psi_m(i)) \quad (15)$$

III. MADDRL APPROACH FOR COMPUTATION OFFLOADING IN NTN-EMPOWERED MEC SYSTEM

We model the optimization problem as MADDRL, where agents learn optimal policies in a decentralized manner. Each UE acts as an agent and as it does not have full observability of the environment, the problem is modelled as Dec-POMDP, which is defined as $(N, S, (A_m)_{m \in N}, P, (R_m)_{m \in N}, \gamma, (O_m)_{m \in N})$, where N indicates the number of UEs, S is the state space, γ is the discount factor, P indicates the transition probability function. O_m , A_m and R_m denotes the observation space, action space and reward function for m^{th} UE. We aim to solve the problem using a decentralized approach, where each UE makes a local observation of the environment (o_m) which includes information about its location, UAVs locations, task size, waiting time and load level at each edge server. Based on this observation, the UE selects an action (a_m) to determine the task processing location and receives a reward (r_m) in terms of cost. Therefore, we design two distributed approaches, utilizing CTDE-based framework and IL-based framework.

We propose the computation offloading algorithm based on dueling DQN model [11], augmented with target Q-network and experience replay buffer. The experience replay buffer stores past experiences for neural network training. The primary network is trained to produce the optimal state-action value. The target network which has the same neural network

structure as that of primary network is periodically updated by copying weights from the primary network to stabilize the training process and facilitate effective learning and convergence. Furthermore, the neural network in the last layer is split into two components: the first estimates state values ($V(s)$), indicating expected rewards, while the second evaluates action advantages ($A(s, a)$), quantifying action superiority. These components collaborate to derive Q-values for state-action pairs, given by $Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a' \in A} A(s, a')$.

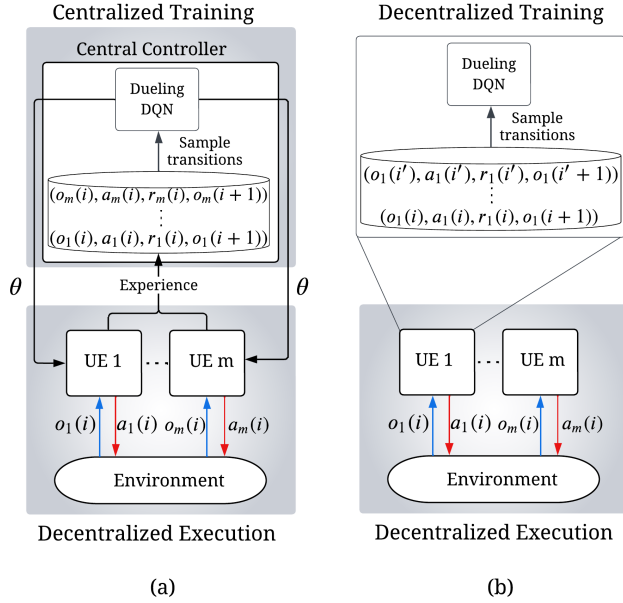


Fig. 1: MADDRL approach: (a) CTDE-based; (b) IL-based.

A. CTDE-based Computation offloading

The proposed CTDE-based framework consists of three key phases: information generation, centralized training and decentralized execution.

Information Generation Phase: Initially, at each time slot i , each UE m randomly decides whether to process its task locally or offload it to the edge server deployed on a UAV or LEO satellite. It records its local observation of the environment, including its location ($\mathbf{p}_m(i)$), UAV locations ($\mathbf{q}_1(i), \dots, \mathbf{q}_n(i)$), and its task size ($D_m(i)$). It calculates queue information including its queue length at each edge node ($q_{m,n}^{edge}(i-1)$, $n \in \mathcal{N}$), waiting times for local computation ($\delta_m^{comp}(i)$) and waiting time for transmission ($\delta_m^{tran}(i)$). Additionally, each edge node broadcast its number of active queues, incurring minimal signaling overhead. UEs store this information to create a history of load levels at edge node $\mathbf{H}(\mathbf{i})$, which is passed through Gated Recurrent Unit (GRU) neural network [12] to predict future load levels.

Centralized Training Phase: In this phase, we train the model based on the proposed framework shown in Fig. 1a. The state information of each UE is described in the information generation phase. Using this information an action is selected which includes the decision regarding where the processing of the task generated by UE m in the time slot i is performed

($\alpha_{m,c}(i)$). If $\alpha_{m,c}(i) = 1$ indicates that the task generated in time slot i at UE m will be processed at the location c , while 0 indicates otherwise. Finally, the reward function is designed to be negatively correlated to the cost ($-\Psi_m(i)$) if the task is successfully processed before the deadline. However, a penalty is applied, if the task is dropped.

At each time step, each UE m observes its local information, takes action, obtains a reward and transitions to next state, generating an experience tuple ($o_m(i), a_m(i), r_m(i), o_m(i+1)$). These experiences are sent to the central controller and stored in its replay buffer. A dueling DQN model at the controller is trained by sampling mini-batches of these experiences and updating its weights. After a certain number of training episodes, updated weights are sent to all UEs. Each UE maintains a policy network with the same neural network structure as the primary network in the central controller. UEs update their local model by copying the weights obtained from the central controller which are then used to decide the actions and generate more refined information.

Decentralized Execution Phase: After deploying a well-trained dueling DQN at the UEs. Each UE can make a decentralized computation offloading decision based on its local information without a central controller. The complete process is explained in the Algorithm 1.

Algorithm 1: CTDE-based Computation Offloading

- 1 Initialize primary and target network of central controller with random weights θ, θ' , where $\theta' = \theta$.
 - 2 Initialize the replay buffer D at the central controller.
 - 3 **for each episode** e **do**
 - 4 **for each time step** i **do**
 - 5 **for each UE** m **do**
 - 6 Obtain observation $o_m(i)$
 - 7 Choose action $a_m(i)$
 - 8 Execute joint action
 $a(i) = \{a_1(i), a_2(i), \dots, a_m(i)\}$
 - 9 **for each UE** m **do**
 - 10 Receive reward $r_m(i)$
 - 11 Obtain next observation $o_m(i+1)$
 - 12 Store transition
 $(o_m(i), a_m(i), r_m(i), o_m(i+1))$ in D
 - 13 Randomly sample a mini-batch of transitions from D
 - 14 Use sampled transitions to train the primary network and update its weights θ
 - 15 Every c time steps, update target network weights $\theta' = \theta$
 - 16 **for each UE** m **do**
 - 17 Every x time steps, copy weights from primary network to policy network
-

B. IL-based framework

This framework consists of three phases: information generation, decentralized training and decentralized execution. The information generation and decentralized execution phase are same as discussed above. However, it differs in the way the training is performed.

Decentralized Training Phase: As shown in Figure 1b, this framework follows a decentralized training. Similar to the previous framework, same dueling DQN model is deployed at each UE and the training is performed locally without a central controller. At each time step i , each UE m generates an experience tuple $(o_m(i), a_m(i), r_m(i), o_m(i+1))$ and stores it in its local replay buffer and trains the model independently. The overall process is described in Algorithm 2.

Algorithm 2: IL-based Computation Offloading

```

1 Initialize primary and target network for all UEs with
  random weights  $\theta_m, \theta'_m$ , where  $\theta'_m = \theta_m$ 
2 Initialize the replay buffer  $D_m$  for all UE  $m$ 
3 for each episode  $e$  do
4   for each time step  $i$  do
5     for each UE  $m$  do
6       Obtain observation  $o_m(i)$ 
7       Choose action  $a_m(i)$ 
8       Receive reward  $r_m(i)$ 
9       Obtain next observation  $o_m(i+1)$ 
10      Store transition
         $(o_m(i), a_m(i), r_m(i), o_m(i+1))$  in  $D_m$ 
11      Randomly sample a mini-batch of transition
        from  $D_m$ 
12      Use sampled transitions to train the primary
        network and update its weights  $\theta_m$ 
13  Every  $c$  time steps, update target network weights
     $\theta'_m \leftarrow \theta_m$ 

```

IV. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our proposed approach. The considered environment is of 100 m x 100 m area with 40 mobile UEs, 4 UAVs and a LEO satellite constellation. Additional simulation parameters are summarized in Table I. The model is trained for 1000 episodes, with each episode comprising 100 time slots of 0.2 seconds each. We

TABLE I: Simulation parameters for the system model

Parameter	Value	Parameter	Value	Parameter	Value
M, N	40, 4	$B_{m,n}$	1 MHz	L, W, H	100, 100, 100 m
Δ	0.2s	P_{NLOS}	20 dB	σ^2	-10^2 dBm
I	100	$P_{up,n}$	0.1 W	D_m	2 - 5 Mbits
w_1	0.5	$P_{up,s}$	0.3 W	τ_m	10 timeslots
w_2	0.5	$r_{m,s}$	14 Mbps	v	1 m/s
P	0.3	j	2	t_{mov}	1 s
s_m	0.297	f_n^{edge}	2.5 GHz (UAV), 15 GHz (satellite)		
	gigacycles/Mbits				

utilize the RMSProp optimizer and learning rate, discount factor, and batch size are set to 0.001, 0.9, and 32, respectively.

We define two baseline approaches for performance comparison: (i) *Random policy (Random)*, where each UE randomly makes the decision to either process the task locally or offload it to satellite or UAV and (ii) *Local policy (Local)*, where UEs never offloads their task and process them locally.

Figure 2 compares the proposed approaches with baseline methods. MADDRL approaches achieve significantly lower average cost, highlighting its advantage over local and random policies. It is observed that the CTDE-based framework outperforms IL-based framework, local policy, and random policy by 11%, 42% and 66% in terms of average cost, respectively.

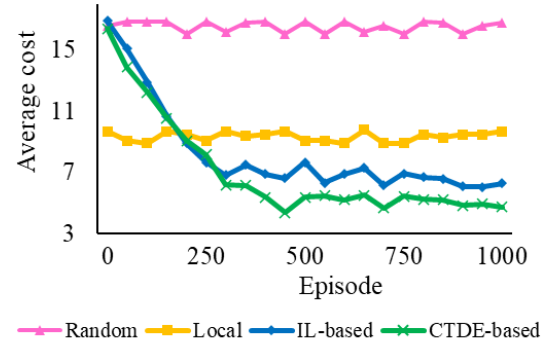


Fig. 2: Performance comparison of the algorithms for average cost

Figures 3a to 3f, illustrate performance comparisons under different simulation settings, consistently showing the superiority of the proposed MADDRL approaches over baseline methods. Furthermore, the CTDE-based framework performs better than the IL-based framework. In Fig. 3a, comparing with different number of UEs, CTDE-based framework exhibits 7%-23% improvement over IL-based framework for UEs ranging from 10 to 50. Similarly, in Fig. 3b, CTDE-based framework show enhancements of 23%, 12%, and 15% for task generation rates of 0.1, 0.3, and 0.5, respectively, compared to IL-based framework. Furthermore, Figs. 3c and 3d demonstrate CTDE's consistent achievement of lower average costs across various task sizes and deadlines, with improvements of 9%-13% and 5%-10%, respectively, compared to IL-based framework. Moreover, in Figs. 3e and 3f, CTDE-based framework showcases performance improvements of 5%-16% and 9%-36% over IL-based framework when considering different penalty value and weighting factors, respectively. These findings highlight the effectiveness of CTDE-based framework in achieving better average cost compared to both baseline and IL-based framework.

Furthermore, to evaluate the learning effectiveness of the proposed approaches, we analyze it across two key metrics: (i) *Convergence rate*, which measures the number of episodes required for the model during the training to reach to the state of convergence, (ii) *Training time*, which refers to the clock time required to train both distributed models on the same computer

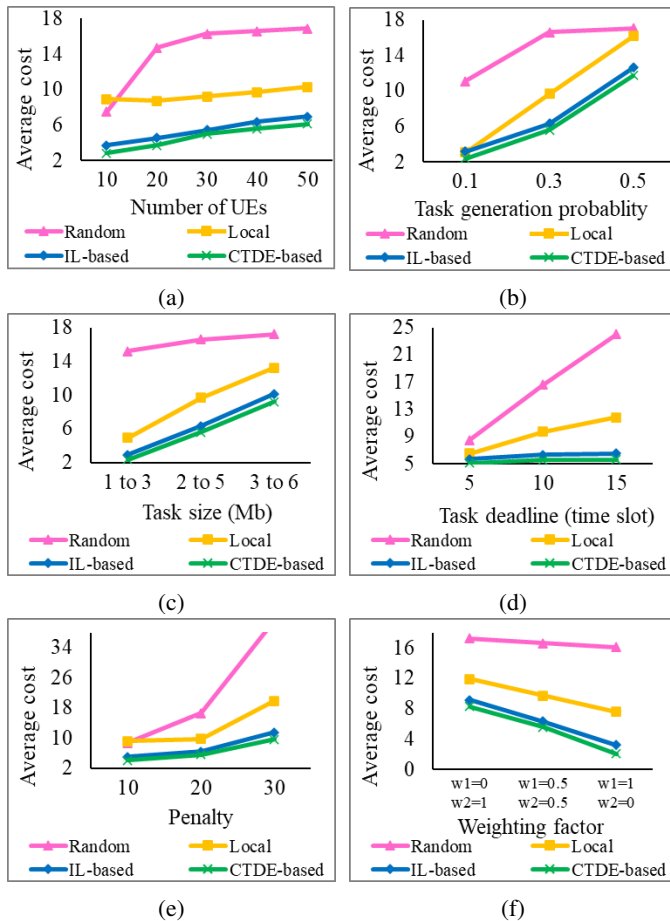


Fig. 3: Comparison of the proposed approaches with other baseline methods with different (a) number of UEs, (b) task generation probability, (c) task size, (d) task deadline, (e) penalty, and (f) weighting factor.

under the same setting. Figure 2 demonstrates that the IL-based framework outperforms the CTDE-based framework in terms of convergence rate, requiring approximately 237 episodes compared to CTDE's 393 episodes. Moreover, the IL-based framework completes the training of 1000 episodes in 1677 seconds, whereas CTDE takes 2358 seconds. This highlights the efficiency of the IL-based framework, as it achieves faster convergence and shorter training times compared to the CTDE-based framework.

The above results highlight that the CTDE-based framework achieves superior results compared to the IL-based framework in terms of average cost due to its utilization of asynchronous experiences sent to the central controller by the UEs. However, it requires longer convergence time and higher training time due to centralized training. It also suffers from increased communication overhead due to information exchange between UEs and the central controller. Therefore, it can be concluded that for resource-constrained UEs, a central controller with high computation power can be installed, but it suffers from communication overhead and delayed training. Alternatively,

UEs with sufficient computation capacity can adopt IL-based framework for near-optimal solutions.

V. CONCLUSION

This paper proposed MADDRL approaches, including CTDE-based and IL-based frameworks, for designing computation offloading strategies in NTN-empowered MEC systems. The evaluation results demonstrate that the CTDE-based framework outperforms IL-based framework, local policy, and random policy by 11%, 42% and 66% in terms of average cost, respectively. Conversely, the IL-based framework demonstrate advantages over the CTDE-based framework by approximately 39% and 28% in terms on convergence rate and training time. This research underscores the efficacy of employing MADDRL approaches in optimizing computation offloading strategies within NTN-empowered MEC systems. Future work could explore a networked agents framework in MADDRL, enabling limited inter-agent communication for more optimal policies. Additionally, investigating partial computation offloading, where tasks are divided between local processing and offloading to edge servers, offers a promising direction.

REFERENCES

- [1] N. Fatima, P. Saxena, and M. Gupta, "Integration of multi access edge computing with unmanned aerial vehicles: Current techniques, open issues and research directions," *Physical Communication*, vol. 52, p. 101641, 2022.
- [2] X. Chen, C. Wu, Z. Liu, N. Zhang, and Y. Ji, "Computation offloading in beyond 5g networks: A distributed learning framework and applications," *IEEE Wireless Communications*, vol. 28, no. 2, pp. 56–62, 2021.
- [3] Y. Chen, B. Ai, Y. Niu, H. Zhang, and Z. Han, "Energy-constrained computation offloading in space-air-ground integrated networks using distributionally robust optimization," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 11, pp. 12 113–12 125, 2021.
- [4] N. Fatima, P. Saxena, and G. Giambene, "Deep reinforcement learning based computation offloading for xurllc services with uav-assisted iot-based multi-access edge computing system," *Wireless Networks*, pp. 1–17, 2023.
- [5] C. Zhou, W. Wu, H. He, P. Yang, F. Lyu, N. Cheng, and X. Shen, "Delay-aware iot task scheduling in space-air-ground integrated network," in *2019 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2019, pp. 1–6.
- [6] F. Xu, F. Yang, C. Zhao, and S. Wu, "Deep reinforcement learning based joint edge resource management in maritime network," *China Communications*, vol. 17, no. 5, pp. 211–222, 2020.
- [7] Z. Wang, H. Yu, S. Zhu, and B. Yang, "Curriculum reinforcement learning-based computation offloading approach in space-air-ground integrated network," in *2021 13th International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, 2021, pp. 1–6.
- [8] S. Zhang, A. Liu, C. Han, X. Liang, X. Xu, and G. Wang, "Multi-agent reinforcement learning-based orbital edge offloading in sagin supporting internet of remote things," *IEEE Internet of Things Journal*, 2023.
- [9] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," *Handbook of reinforcement learning and control*, pp. 321–384, 2021.
- [10] K. Yu, Q. Cui, Z. Zhang, X. Huang, X. Zhang, and X. Tao, "Efficient uav/satellite-assisted iot task offloading: A multi-agent reinforcement learning solution," in *2022 27th Asia Pacific Conference on Communications (APCC)*. IEEE, 2022, pp. 83–88.
- [11] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International conference on machine learning*. PMLR, 2016, pp. 1995–2003.
- [12] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.